arXiv:submit/7363561 [cs.AI] 14 Mar 2026

Trained Persistent Memory for Frozen Encoder–Decoder LLMs: Six Architectural Methods

Hong Jeong
Inha University in Tashkent, Uzbekistan
hjeong@postech.ac.kr

Abstract

Frozen encoder–decoder language models are stateless: the latent representation is discarded after every forward pass, so no information persists across sessions. We present six trained architectural methods for adding persistent memory *in the continuous latent space* of a frozen backbone while keeping all encoder and decoder weights fixed. Unlike text-level memory systems that store and retrieve string summaries, our memory bank lives entirely in the encoder's representation space, so every write and read is a differentiable operation on dense vectors. The methods span three injection points (before the encoder, between encoder and decoder, inside the decoder) and four write mechanisms (attention-coupled update, Hebbian outer product, gated cross-attention, sparse slot addressing). After training, the memory bank continues to accumulate at inference time without gradients, enabling *conversational learning*. Under an absolute forgetting-curve evaluation on LoCoMo, the stateless baseline scores exactly zero, all six trained adapters produce positive retained-memory curves, Slot memory is best overall (3.83% across all lag buckets), and Gated memory is strongest at short lag (4.29%) but decays to 2.25% beyond 256 turns.

1 Introduction

Consider a frozen encoder–decoder model such as Flan-T5 built on a T5-style backbone [1]. The forward pass is:

$$Z_t = E_{\text{frozen}}(x_t), \qquad \hat{y}_t = D_{\text{frozen}}(Z_t), \tag{1}$$

where E_{frozen} and D_{frozen} are fixed pre-trained weights, x_t is the input at turn t, $Z_t \in \mathbb{R}^{n \times d}$ is the encoder output, and $\hat{y}_t$ is the generated text. This system is **stateless**: Z_t is discarded after each forward pass and the model has no recollection of previous turns. If a user says "I like reading" in session 1 and asks "What do I like?" in session 3, the model cannot answer—there is no state that survives across sessions. This *inter-session memory* problem is the concrete target of persistent memory.

Existing long-term memory solutions such as MemGPT and MemoryBank operate at the *text level*: they store, summarize, and retrieve natural-language passages through an external database. This paper works at a fundamentally different level—the **latent space** of the frozen model. The memory bank $P_t \in \mathbb{R}^{n_P \times d}$ holds continuous encoder representations, not strings, so writing and reading are differentiable operations embedded inside the forward pass rather than pre- or post-processing steps around it.

A persistent bank built directly from frozen encoder outputs is not enough. The decoder's cross-attention was pre-trained to read current encoder states, not arbitrarily accumulated cache states, so a naive memory path tends to dilute attention rather than sharpen retrieval as history grows. Recent

Preprint.

attention-coupled latent-memory work shows that learned structure inside the memory pathway can instead induce functional specialization and controlled routing [6, 7]. For frozen encoder–decoder LLMs, this suggests that a small trainable adapter is the minimal mechanism needed to write memory in a form the frozen decoder can use.

This paper takes the necessary next step: we **allow training** for a small memory adapter θ_{Mem} while keeping both encoder and decoder frozen. We augment the stateless system with a **persistent memory bank** $P_t \in \mathbb{R}^{n_P \times d}$ that persists across turns and sessions:

$$Z_t = E_{\text{frozen}}(x_t), \qquad P_t = \text{Write}(P_{t-1}, Z_t), \qquad \hat{y}_t = D_{\text{frozen}}\big(\text{Read}(Z_t, P_{t-1})\big). \tag{2}$$

The `Write` operation updates P from the current latent; the `Read` operation injects historical context from P into the decoder. The learned parameters θ_{Mem} enable the adapter to *learn how to write memory in a format that the frozen decoder's existing cross-attention can discriminate*—the capability that zero-training methods provably lack.

Empirically, this is enough to separate the six designs. All trained adapters rise above the zero baseline on the absolute forgetting curve, M.6 Slot is strongest overall, and M.5 Gated trades the highest short-lag write strength for faster long-lag decay.

After training, θ_{Mem} is frozen but P_t continues to accumulate at inference time without gradients. We call this **conversational learning**: each new session enriches P, so a fact stated in session 1 ("I am John Doe") can be recalled in session 10 ("Who am I?") without re-stating it and without million-token context windows—the relevant inter-session history is compressed into P.

The analogy to human cognition is deliberate. Human brains accumulate knowledge through complementary memory systems [8]: *episodic* memory records specific events ("Alice mentioned a trip to Paris"), *semantic* memory distills general facts ("Alice likes travel"), *procedural* memory encodes how-to skills, and *working memory* holds the active conversational context. All of these are linked through *associative* retrieval: a cue in working memory can trigger recall from any long-term store. The persistent memory bank P in our framework plays an analogous role: the write rule determines what is stored (episodic vs. semantic), the read rule implements associative retrieval, and the capacity of P constrains how much can be retained—mirroring the interplay of encoding, consolidation, and retrieval in biological memory. A successful method must demonstrate not merely storage but *remembering* (retrieving the right fact), *generalisation* (answering questions phrased differently from the original statement), and *abstraction* (combining multiple facts into a coherent response)—the hallmarks of genuine learning from experience.

The six methods differ along three orthogonal design dimensions: (i) *where* P enters the forward pass (before the encoder, between encoder and decoder, or inside the decoder), (ii) *how* P is written (attention-coupled update, Hebbian outer product, gated cross-attention, or sparse slot addressing), and (iii) *how many parameters are added* (all modest relative to the 248M-parameter backbone). A crucial constraint across all methods is that the frozen decoder is calibrated exclusively for encoder outputs. Let $\mathcal{M}_E$ denote the set of representations the encoder can produce; any method that replaces Z with $H \notin \mathcal{M}_E$ will degrade the decoder. All six methods preserve the frozen encoder–decoder route through Z_t and ensure that memory influence enters through controlled, learnable pathways.

Our core evaluation is simple: **how much does each method remember?** Every condition—baseline and all six memory methods—sees only the current turn x_t; no method receives the conversation history. The baseline is deliberately short-sighted and retains nothing. Each memory method must accumulate facts into P through its write rule and retrieve them through its read path. By probing the same factual questions across increasing evidence lag, we measure an *absolute forgetting curve*: how much answer quality is lost when a method's own persistent state is ablated. The stateless baseline has no persistent state and is therefore identically zero; stronger methods start higher at short lag and decay more slowly as the evidence recedes into the past.

A secondary question is whether the frozen decoder's cross-attention queries—trained only on encoder outputs—have enough **representational slack** to attend usefully to memory entries projected by the trained adapter. If not, even perfectly trained θ_{Mem} will not help, implying that the frozen decoder itself is the bottleneck.

Contributions.

1. **Latent-space persistent memory.** We formulate the problem of adding persistent memory that lives entirely in the continuous latent space of a frozen encoder–decoder LLM. Unlike text-level memory systems (MemGPT, MemoryBank) that store and retrieve natural-language strings outside the model, our memory bank $P_t \in \mathbb{R}^{n_P \times d}$ holds dense encoder representations; every write and read is a differentiable operation inside the forward pass.
2. **Six architectural methods.** We design, implement, and release six trained memory adapters that span three injection points (before the encoder, between encoder and decoder, inside the decoder) and four write mechanisms (attention-coupled update, Hebbian outer product, gated cross-attention, sparse slot addressing). All methods keep every encoder and decoder weight frozen and add only a small set of learnable parameters θ_{Mem}.
3. **Absolute forgetting-curve evaluation.** We introduce an evaluation protocol that measures how much answer quality is lost when a method's own persistent state is ablated, as a function of evidence lag. The stateless baseline is identically zero by construction, giving a principled anchor for comparison.
4. **Empirical findings.** Under this protocol on LoCoMo, all six trained adapters produce positive retained-memory curves. M.6 Slot is best overall (3.83 %); M.5 Gated has the strongest short-lag write signal (4.29 %) but the fastest long-lag decay (2.25 % beyond 256 turns). The results also confirm that the frozen decoder possesses sufficient representational slack to attend usefully to adapter-projected memory entries.

2 Related Work

Persistent memory is adjacent to, but distinct from, several existing lines of work. Application-level long-term memory systems such as MemGPT [2] and MemoryBank [3] demonstrate that explicit memory can improve LLM behaviour over extended interactions, but they operate at the *text level*: facts are stored as natural-language strings, retrieval is a search over those strings, and the language model itself is unchanged. LoCoMo [5] provides a public benchmark targeted specifically at very long-term conversational memory and motivates the multi-session evaluation setting adopted here.

The present paper operates at a fundamentally different level—the **latent space** of the frozen model. Our persistent memory bank stores continuous encoder representations, not text; reading and writing are differentiable operations inside the forward pass rather than pre- or post-processing steps. Rather than proposing a single end-to-end memory agent, we define a taxonomy of six architectural alternatives for latent-space persistent memory, formalize their read and write paths, and specify a public-dataset protocol for comparing them against a stateless baseline under a shared released backbone.

Parameter-efficient adaptation. Several of our six methods adapt ideas originally proposed for parameter-efficient fine-tuning or memory-augmented architectures. Prefix tuning [11] prepends learnable soft tokens to the input; M.1 extends this idea to a persistent memory bank. Flamingo [12] inserts gated cross-attention layers into a frozen decoder for visual grounding; M.2 and M.5 adopt the same parallel-branch topology for memory injection. Memorizing Transformers [13] extend the decoder KV cache with retrieved past representations; M.3 follows the same KV concatenation principle. Linear Transformers and fast weight programmers [14] accumulate an outer-product associative matrix updated at every step; M.4 uses the same Hebbian write rule. Neural Turing Machines [15] maintain addressable memory slots with content-based sparse writes; M.6 inherits this slot-addressing mechanism. Crucially, none of these prior methods were designed for *persistent latent-space* memory that accumulates across sessions inside a frozen encoder–decoder model. Our contribution is not the individual read or write primitive but the controlled comparison—under a single frozen backbone and a common forgetting-curve evaluation—of how these primitives perform as latent-space persistent memory for inter-session recall.

Attention-coupled latent memory. Recent arXiv work on attention-coupled latent memory explores richer structured bank dynamics than the minimal adapters studied here. Inhibitory cross-talk across paired banks can drive functional lateralization [6], while a miniature brain-transformer architecture adds thalamic gating, amygdaloid salience, hippocampal lateralization, and prefrontal working memory to shape routing and consolidation [7]. The present paper is narrower: it compares six simpler trainable memory adapters under a fixed frozen encoder–decoder backbone and a common forgetting-curve evaluation.

Cognitive memory systems. Our taxonomy is informed by the cognitive science of human memory. Tulving's distinction between episodic and semantic memory [8] maps onto our design choices: methods with high-capacity slot banks (M.6) store episode-like snapshots, while Hebbian associative memory (M.4) naturally distils co-occurrence statistics akin to semantic memory. The content-gated method (M.5) resembles working-memory gating, selectively admitting relevant information. Modern complementary learning systems theory [9, 10] argues that fast episodic binding and slow semantic consolidation are both necessary; our experiment measures whether any single-mechanism method suffices or whether the task demands a composite architecture.

3 Problem Setting and Notation

Figure 1 shows the stateless frozen encoder–decoder pipeline used as the control architecture throughout the paper.

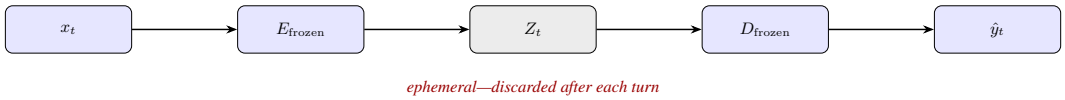

Figure 1: Frozen encoder–decoder baseline used as the stateless control. The latent representation is consumed within the current turn and then discarded.

The encoder maps x_t to $Z_t \in \mathbb{R}^{n \times d}$ (sequence length n, hidden dimension d). The decoder uses cross-attention to read Z_t at every layer. Both E_{frozen} and D_{frozen} are frozen; only the persistent memory P and (optionally) a small set of adapter parameters θ_{Mem} are modified.

4 Trained Alternatives

These methods introduce a small set of learnable parameters θ_{Mem} trained via backpropagation; at inference time (Type 2), P continues to accumulate without gradients.

Every method uses *content-based* addressing—deciding what to store or retrieve based on the semantic content of the current turn rather than fixed positions. Table 1 gives the complete read and write operations for all six methods; the baseline (M.0) is included for reference.

Table 1: Read and write operations for each method. $A = \text{softmax}(Z_t W_Q (P W_K)^\top / \sqrt{d})$. *Delegated* read means the frozen decoder cross-attention selects from the concatenated KV; *explicit* read means the adapter performs its own retrieval before passing to the decoder.

	Method	**Write rule** ($P_t \leftarrow$)	**Read rule** (inject into decoder)	**Type**
0	Baseline	—	$D(Z_t)$	—
1	Prefix	$\gamma P + A^\top V,\ V{=}Z_t W_V$	$[Z_t;\ P\,W_P] \to D$ (extra KV)	Delegated
2	XAttn	$\gamma P + A^\top V,\ V{=}Z_t W_V$	$\text{softmax}(s\,W_Q^m (P\,W_K^m)^\top / \sqrt{d})\,(P\,W_V^m)$	Explicit
3	KV Extension	$\gamma P + A^\top V,\ V{=}Z_t W_V$	$K{=}[K_Z;\ PW_{K,m}],\ V{=}[V_Z;\ PW_{V,m}]$	Delegated
4	Hebbian	$\gamma M + (Z_t W_{K,H})^\top (Z_t W_{V,H})$	$(Z_t W_{Q,H})\,M \to W_{\text{Mem}} \to D$ (extra KV)	Explicit
5	Gated	$\gamma P + A^\top V,\ V{=}Z_t W_V$	$g_t \odot \text{XAttn}(s, P),\ g_t{=}\sigma(W_g[s; c] + b_g)$	Explicit
6	Slot	top-k: $\gamma P[s] + (1{-}\gamma)\,\bar{z}_t W_u$	$K{=}[K_Z;\ PW_{K,m}],\ V{=}[V_Z;\ PW_{V,m}]$	Delegated

Delegated-read methods (M.1, M.3, M.6) project all of P into the decoder's KV cache and let the frozen cross-attention select relevant entries. *Explicit-read* methods (M.2, M.4, M.5) perform their own content-based retrieval before passing the result to the decoder. This distinction has implications for trainability: explicit-read methods introduce more parameters but give the adapter direct control over retrieval; delegated-read methods rely on the decoder's pre-trained attention patterns, which are already tuned for cross-attention selection.

4.1 M. 1: Memory as Encoder-Input Prefix

Persistent memory is compressed into m soft tokens and prepended to the encoder input, extending the prefix-tuning idea [11] from static task prompts to a dynamic, accumulating memory bank. The encoder integrates memory and current input through self-attention and produces a valid $Z \in \mathcal{M}_E$. The decoder remains entirely untouched. Concretely,

$$S_t = U_P P_{t-1} W_P \in \mathbb{R}^{m \times d}, \tag{3}$$

$$\tilde{x}_t = [\ \underbrace{S_t}_{m \text{ soft tokens}}\ ;\ x_t\ ] \in \mathbb{R}^{(m+n) \times d}, \tag{4}$$

$$\tilde{Z}_t = E_{\text{frozen}}(\tilde{x}_t), \qquad Z_t = \tilde{Z}_t[m{:}], \tag{5}$$

$$\hat{y}_t = D_{\text{frozen}}(Z_t), \tag{6}$$

where $U_P \in \mathbb{R}^{m \times n_P}$ mixes the n_P memory rows into m prefix slots and $W_P \in \mathbb{R}^{d \times d}$ is a learnable feature projection. If $n_P = m$, one may set $U_P = I$, reducing the prefix to $S_t = P_{t-1} W_P$.

Memory is updated via an attention-coupled write rule that injects the current turn's content into the memory bank:

$$\begin{aligned} Q &= Z_t W_Q, \quad K = P_{t-1} W_K, \quad V = Z_t W_V, \\ A_t &= \operatorname{softmax}\!\big(Q K^\top / \sqrt{d}\big), \qquad P_t = \gamma P_{t-1} + A_t^\top V. \end{aligned} \tag{7}$$

Q and K perform content-based addressing between the current latent Z_t and the existing memory P_{t-1}. Crucially, values V are drawn from Z_t, not from P; $A_t^\top V$ aggregates the current turn's content into n_P memory rows, weighted by the addressing scores. This ensures that new information enters memory at every turn—without it, an all-zero initialisation would remain zero indefinitely.

Figure 2 illustrates how memory is projected into a soft prefix before the frozen encoder, with the current latent driving a write-back update.

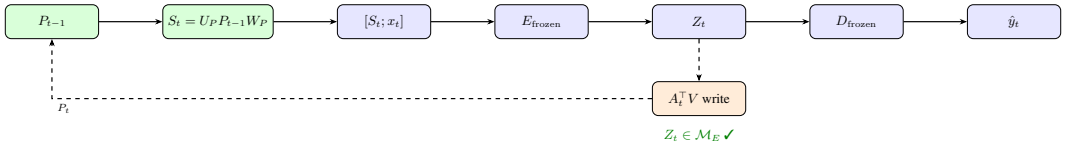

Figure 2: M. 1 injects persistent memory as an encoder-input prefix and writes the current latent back into memory through an attention-coupled update.

The learnable parameters are $\theta_{\text{Mem}} = \{U_P, W_P, W_Q, W_K, W_V, \gamma\}$. Under Type 1 training, θ_{Mem} is optimised via $\nabla_{\theta_{\text{Mem}}} \mathcal{L}$; under Type 2, P_t accumulates at inference with θ_{Mem} frozen.

4.2 M. 2: Parallel Decoder Cross-Attention

A parallel cross-attention layer is inserted in each decoder block to attend to P independently of the frozen pathway, following the Flamingo architecture [12] which showed that interleaved cross-attention can inject external information into a frozen LM. The original Z route is untouched; memory influence is *additive* via a zero-initialised coefficient. This method requires source-level access to the decoder blocks of an open-weight model. Here *frozen* means that the original decoder weights are not updated; it does not mean that intermediate layers are inaccessible.

At decoder layer ℓ with hidden state $s_t^{(\ell)}$:

$$c_{\text{Mem}}^{(\ell)} = \text{XAttn}_{\text{Mem}}^{(\ell)}\big(s_t^{(\ell)},\ P_{t-1}\big), \tag{8}$$

$$s_t^{(\ell)\prime} = s_t^{(\ell)} + \text{XAttn}_{\text{frozen}}^{(\ell)}\big(s_t^{(\ell)},\ Z_t\big) + \underbrace{\beta^{(\ell)}}_{\text{init}=0}\ c_{\text{Mem}}^{(\ell)}. \tag{9}$$

At initialisation $\beta^{(\ell)} = 0$, so the model falls back exactly to the frozen baseline. Memory is updated with the same attention-coupled write rule as M.1:

$$Q = Z_t W_Q,\ K = P_{t-1} W_K,\ V = Z_t W_V, \quad P_t = \gamma\, P_{t-1} + A_t^\top V, \tag{10}$$

where $A_t = \mathrm{softmax}(QK^\top/\sqrt{d})$ and V is sourced from Z_t so that new content enters memory regardless of P's current state. Figure 3 illustrates the frozen cross-attention path running in parallel with the additive memory branch.

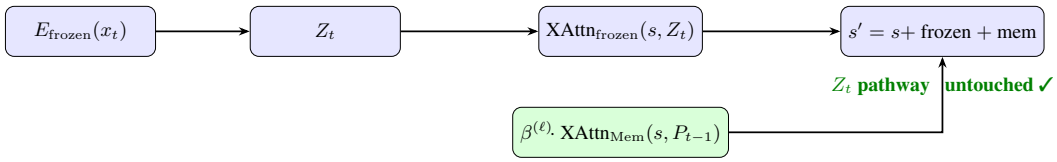

Figure 3: M. 2 preserves the frozen cross-attention route over the current encoder latent and adds a parallel decoder memory branch scaled by a learned coefficient.

The per-layer parameters are $\theta_{\text{Mem}} = \{W_Q^{\text{Mem}}, W_K^{\text{Mem}}, W_V^{\text{Mem}}, O^{\text{Mem}}, \beta^{(\ell)}\}$, totalling $\sim$4.1M across all 12 decoder layers of Flan-T5-base (1.7%).

4.3 M. 3: Decoder KV Extension

Persistent memory is projected into additional key–value pairs that are concatenated alongside Z in the decoder's cross-attention, following the KV-extension principle of Memorizing Transformers [13], and leaving the original Z positions byte-for-byte preserved. At decoder cross-attention layer ℓ:

$$K^{(\ell)} = \left[\, W_K^{(\ell)} Z_t \;;\; W_{K,\text{Mem}}^{(\ell)} P_{t-1} \,\right] \in \mathbb{R}^{(n+n_P)\times d_k}, \tag{11}$$

$$V^{(\ell)} = \left[\, W_V^{(\ell)} Z_t \;;\; W_{V,\text{Mem}}^{(\ell)} P_{t-1} \,\right] \in \mathbb{R}^{(n+n_P)\times d_v}, \tag{12}$$

$$\hat{y}_t = D_{\text{frozen}}\big(Q^{(\ell)},\, K^{(\ell)},\, V^{(\ell)}\big). \tag{13}$$

Queries $Q^{(\ell)}$ use the frozen $W_Q^{(\ell)}$. Zero-initialised $W_{K,\text{Mem}}, W_{V,\text{Mem}}$ ensure no-regression at init. The write rule is the same attention-coupled update as M.1 (Eq. (7)):

$$Q = Z_t W_Q, \quad K = P_{t-1} W_K, \quad V = Z_t W_V, \quad P_t = \gamma P_{t-1} + \mathrm{softmax}\big(QK^\top/\sqrt{d}\big)^\top V. \tag{14}$$

Figure 4 shows decoder keys and values extended with memory-derived entries while the current encoder positions remain unchanged.

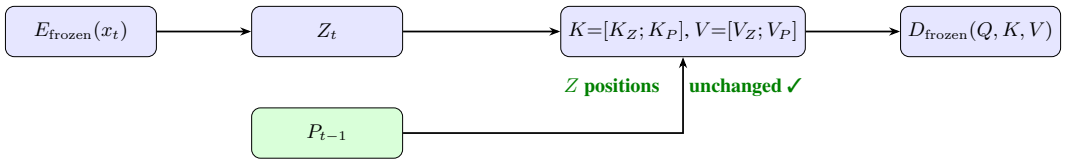

Figure 4: M. 3 extends decoder keys and values with learned projections of persistent memory while keeping the current encoder tokens on their original path.

The learnable parameters are $\theta_{\text{Mem}} = \{W_{K,\text{Mem}}^{(\ell)}, W_{V,\text{Mem}}^{(\ell)}, W_Q, W_K, W_V, \gamma\}_\ell$, totalling $\sim$2.4M across all layers (1.0%).

4.4 M. 4: Hebbian / Associative Memory

An outer-product Hebbian rule—the same write primitive used in linear transformers and fast weight programmers [14]—accumulates associative structure in a matrix M_t, and the full read path is made explicit by injecting the recalled memory through decoder KV extension. This yields a complete, experimentally realizable architecture rather than a generic "inject somehow" formulation. Let d_h

denote the associative-memory dimension:

$$\tilde{M}_t = \gamma M_{t-1} + \frac{1}{n}(Z_t W_{K,H})^\top (Z_t W_{V,H}),$$

$$M_t = \tilde{M}_t \big/ \max\big(\|\tilde{M}_t\|_F,\, 1\big) \;\in \mathbb{R}^{d_h \times d_h}, \tag{15}$$

$$R_t = (Z_t W_{Q,H}) M_{t-1} \in \mathbb{R}^{n \times d_h}, \tag{16}$$

$$K_M^{(\ell)} = R_t W_{K,\mathrm{Mem}}^{(\ell)}, \qquad V_M^{(\ell)} = R_t W_{V,\mathrm{Mem}}^{(\ell)}, \tag{17}$$

$$\hat{y}_t = D_{\mathrm{frozen}}\Big(Q^{(\ell)}, \big[W_K^{(\ell)} Z_t;\; K_M^{(\ell)}\big], \big[W_V^{(\ell)} Z_t;\; V_M^{(\ell)}\big]\Big). \tag{18}$$

Figure 5 shows the Hebbian matrix being read by the current latent and exposed to the decoder as extra key-value memory.

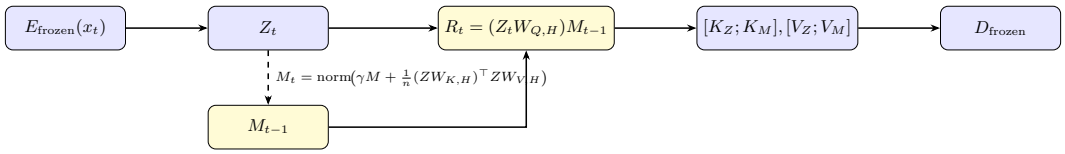

Figure 5: M. 4 stores associative structure in a Hebbian memory matrix that is queried by the current latent and passed to the decoder as additional memory.

The learnable parameters are $\theta_{\mathrm{Mem}} = \{W_{Q,H}, W_{K,H}, W_{V,H}, W_{K,\mathrm{Mem}}^{(\ell)}, W_{V,\mathrm{Mem}}^{(\ell)}, \gamma\}$. W_{Mem} (the read projection from d_h back to d) is zero-initialised so that the memory branch is silent at startup (safe startup). $W_{Q,H}$ uses small random initialisation, $\mathcal{N}(0, 0.02)$: because W_{Mem} is zero, the gradient signal for $W_{Q,H}$ comes from the loss through W_{Mem} once it becomes non-zero, and conversely the gradient for W_{Mem} requires $R_t \neq 0$, i.e. $W_{Q,H} \neq 0$. Zero-initialising *both* $W_{Q,H}$ and W_{Mem} would create a gradient deadlock.

4.5 M. 5: Context-Gated Decoder Memory Branch

Rather than modifying encoder outputs, a lightweight memory branch is inserted *inside the decoder*, using a content-dependent gate inspired by Flamingo's tanh-gated cross-attention [12]. The branch reads from P, and the gate controls how strongly its output affects the decoder hidden state:

$$c_{\mathrm{Mem}}^{(\ell)} = \mathrm{XAttn}_{\mathrm{Mem}}^{(\ell)}\big(s_t^{(\ell)},\, P_{t-1}\big), \tag{19}$$

$$g_t^{(\ell)} = \sigma\big(W_g^{(\ell)}[s_t^{(\ell)}; c_{\mathrm{Mem}}^{(\ell)}] + b_g^{(\ell)}\big), \tag{20}$$

$$s_t^{(\ell)\prime} = s_t^{(\ell)} + \mathrm{XAttn}_{\mathrm{frozen}}^{(\ell)}\big(s_t^{(\ell)}, Z_t\big) + g_t^{(\ell)} \odot c_{\mathrm{Mem}}^{(\ell)}. \tag{21}$$

With $b_g^{(\ell)} < 0$ at initialisation, the memory branch starts nearly off, so the model initially behaves like the frozen baseline and only later opens the memory pathway where helpful. Figure 6 illustrates the decoder-side branch whose contribution is controlled by the learned context gate.

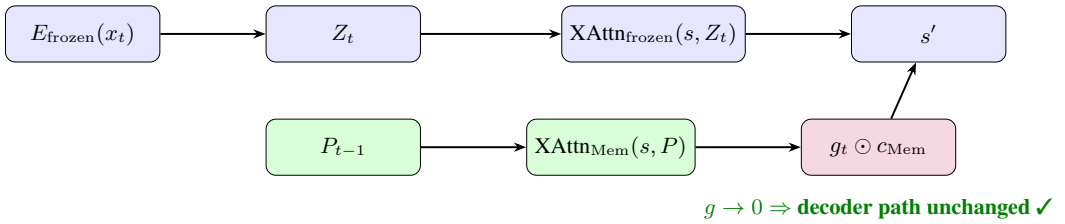

Figure 6: M. 5 adds a memory read branch inside the decoder and lets a learned gate decide when the auxiliary memory signal should influence the frozen path.

The learnable parameters are $\theta_{\mathrm{Mem}} = \{W_Q^{\mathrm{Mem}}, W_K^{\mathrm{Mem}}, W_V^{\mathrm{Mem}}, W_g, b_g\}$ together with the write-side parameters. The write rule is the same attention-coupled update as M.1 (Eq. (7)):

$$Q = Z_t W_Q, \quad K = P_{t-1} W_K, \quad V = Z_t W_V, \quad P_t = \gamma P_{t-1} + \mathrm{softmax}\big(QK^\top/\sqrt{d}\big)^\top V. \tag{22}$$

4.6 M. 6: Slot-Based Memory with Sparse Write

Memory is organised as S fixed-size slots $P \in \mathbb{R}^{S \times d}$, adopting the addressable-slot design of Neural Turing Machines [15]. At each turn, only the top-k addressed slots are updated; the read path is an explicit decoder KV extension:

$$\bar{z}_t = \frac{1}{n} \sum_{i=1}^{n} Z_t[i, :], \tag{23}$$

$$a_t = \mathrm{softmax}\big(\bar{z}_t W_a P_{t-1}^{\top} / \sqrt{d}\big) \in \mathbb{R}^S, \tag{24}$$

$$m_t[s] = \mathbf{1}[s \in \text{top-k}(a_t, k)], \tag{25}$$

$$u_t = \bar{z}_t W_u \in \mathbb{R}^d, \tag{26}$$

$$P_t[s] = (1 - m_t[s]) P_{t-1}[s] + m_t[s] \big(\gamma P_{t-1}[s] + (1-\gamma) u_t\big), \tag{27}$$

$$K^{(\ell)} = \big[W_K^{(\ell)} Z_t;\ P_t W_{K,\mathrm{Mem}}^{(\ell)}\big], \quad V^{(\ell)} = \big[W_V^{(\ell)} Z_t;\ P_t W_{V,\mathrm{Mem}}^{(\ell)}\big]. \tag{28}$$

Figure 7 shows sparse writes into a fixed slot bank that is later read by the decoder as structured episodic memory.

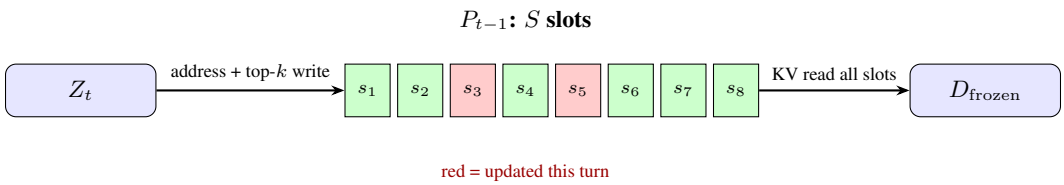

Figure 7: M. 6 writes to a fixed set of memory slots sparsely and exposes the slot bank to the decoder as an explicit episodic memory.

The learnable parameters are $\theta_{\mathrm{Mem}} = \{W_a, W_u, W_{K,\mathrm{Mem}}^{(\ell)}, W_{V,\mathrm{Mem}}^{(\ell)}, \gamma\}$.

5 Training and Inference

Each of the six methods introduces learnable parameters θ_{Mem} trained while both encoder and decoder remain frozen. Training proceeds in two phases.

Gradient flow through frozen networks. "Frozen" means that E and D receive no weight updates—it does *not* mean that gradients cannot flow through them. Both networks act as fixed differentiable functions: in the backward pass the chain rule propagates the loss gradient through the frozen decoder (and, for M. 1, also through the frozen encoder) to reach θ_{Mem}. This is the same principle underlying prefix tuning, LoRA, and adapter methods—the only parameters that receive gradient updates are those belonging to the memory adapter. The frozen backbone's role is to provide both a fixed forward computation and a fixed gradient signal that the adapter must learn to exploit.

In the first phase (**Type 1: supervised learning**), gradients flow from the decoder loss through the read pathway and update θ_{Mem}, thereby learning *how* to read and write memory effectively:

$$\theta_{\mathrm{Mem}} \leftarrow \theta_{\mathrm{Mem}} - \eta \nabla_{\theta_{\mathrm{Mem}}} \mathcal{L}\big(D_{\mathrm{frozen}}(\mathrm{Read}(Z_t, P)),\ y_t\big). \tag{29}$$

Write projections as fixed random maps. During Type 1 training the write rule $P_t = \mathrm{Write}(P_{t-1}, Z_t)$ executes without gradients to prevent the computation graph from growing across the full conversation history. Consequently, the write-side projections (W_Q, W_K, W_V in most methods) receive no gradient updates and remain at their random initialisation throughout training. These projections act as fixed random maps that nonetheless preserve pairwise distances in the input (Johnson–Lindenstrauss property), ensuring that distinct encoder outputs produce distinguishable memory entries. The read-side parameters (W_P, W_{Mem}, cross-attention heads, gates) are therefore the only parameters that receive gradient updates; they learn to decode whatever structure the fixed

write rule deposits in P. Excluding the write-side projections from the optimiser prevents unnecessary weight decay and avoids misleading parameter counts.

In the second phase (**Type 2: conversational learning**), θ_{Mem} is frozen but P_t continues to accumulate. Each conversation enriches P, improving the system's responses without any gradient computation:

$$P_t = \text{Write}(P_{t-1}, Z_t) \quad \text{with } \theta_{\text{Mem}} \text{ fixed.} \tag{30}$$

This is the mechanism we call *conversational learning*: the system becomes more knowledgeable and personalised with every conversation, exactly through repeated online updates while the frozen model weights remain fixed. Figure 8 contrasts the two phases.

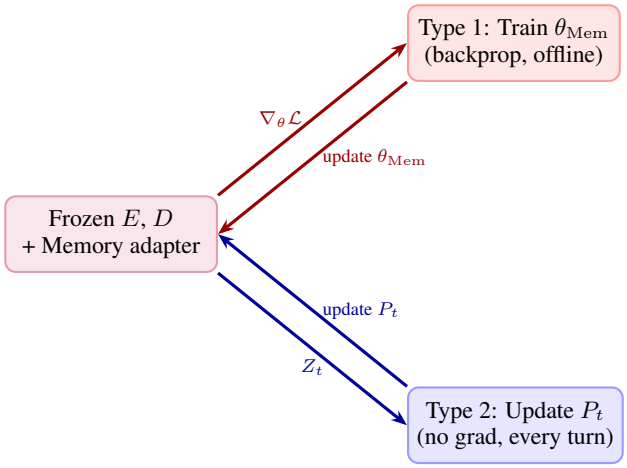

Figure 8: Two learning phases. Type 1 updates memory adapter parameters offline by backpropagation, whereas Type 2 updates the persistent memory online at each turn with frozen model weights.

Table 2 consolidates the six methods along key design dimensions.

Table 2: Six trained persistent-memory methods compared across key design dimensions. "Primary-path safe" means the original frozen encoder–decoder route through Z_t is preserved, even if an auxiliary decoder-side memory branch is added. "Memory cost" is per turn.

	Method	Injection point	Primary-path safe	New params	Mem. cost	Write mechanism
1	Encoder-input prefix	Before E	✓	2.4M	const.	$A_t^\top V$
2	Parallel decoder XAttn	Inside D	✓	4.1M	const.	$A_t^\top V$
3	Decoder KV extension	Inside D xattn	✓	2.4M	const.	$A_t^\top V$
4	Hebbian / associative	Decoder KV	✓	0.8M	$O(d_h^2)$	Hebbian outer prod.
5	Context-gated decoder branch	Inside D	✓	4.7M	const.	gated XAttn
6	Slot-based sparse write	Decoder KV	✓	1.8M	$O(Sd)$	Top-k overwrite

6 Evaluation

The preceding sections specify *how* each method reads and writes persistent memory; this section specifies *what* we measure and *how* we measure it.

6.1 Forgetting-Curve Hypothesis

If persistent memory genuinely converts a stateless model into one that *retains conversation-specific information*, the effect should be measurable as a function of how far in the past the supporting evidence was written. We therefore formalise evaluation around an **absolute forgetting curve** rather than a bag of unrelated scalar metrics.

For a question q asked after T conversational turns, let E_q denote the set of supporting evidence turns and define the evidence lag by the oldest required support:

$$\ell_q = T - \min(E_q). \tag{31}$$

For each trained method m, we compare two answers to the same question: one with the learned persistent state intact, $\hat{y}_q^{\mathrm{mem}}(m)$, and one from the same trained model with its persistent memory state forced to zero, $\hat{y}_q^{0}(m)$. The retained-memory contribution is

$$R_q(m) = \max\Big(0,\ \mathrm{F1}(\hat{y}_q^{\mathrm{mem}}(m), y_q) - \mathrm{F1}(\hat{y}_q^{0}(m), y_q)\Big), \tag{32}$$

where y_q is the gold answer. This quantity is absolute in the sense that it measures how much usable information the method's own persistent memory adds beyond its frozen backbone and trained adapter parameters alone.

For the *stateless baseline*, there is no persistent state to ablate, so the retained-memory score is identically zero for every question:

$$R_q(\text{baseline}) = 0. \tag{33}$$

For an effective stateful system, retained-memory contribution should be largest at short lag and should gradually decrease as the relevant evidence lies further in the past:

$$R_q(m) \text{ should decrease as } \ell_q \text{ increases.} \tag{34}$$

The central experimental prediction is therefore a family of absolute forgetting curves: the baseline is flat at zero by construction, while stronger trained persistent-memory methods start higher at short lag and decay more slowly at long lag. Without learned memory control, retained-memory curves collapse toward zero in the stateless limit; the test is whether trained adapter parameters θ_{Mem} can produce large and durable positive curves.

6.2 Benchmarks and Protocol

Equal-input principle. To isolate the effect of persistent memory, *every* condition—baseline and all six memory methods—receives exactly the same encoder input x_t at each turn: the current conversational turn only. No method is given the full history $[x_0, x_1, \ldots, x_t]$. The baseline is therefore intentionally *short-sighted*: it encodes only the present turn and decodes an answer with no access to prior sessions. Memory methods receive the same x_t but additionally condition the decoder on the persistent state P_{t-1} accumulated from all earlier turns. Any non-zero retained-memory score is thus attributable solely to the persistent state P.

All conditions use the same released frozen encoder–decoder backbone, the same tokenizer, and the same decoding rule. Each example is processed turn by turn: after every conversational turn the method-specific write rule updates P_t; when a question is asked the model answers using the current query and the persistent memory state P_{t-1}. The stateless baseline uses the identical backbone but no persistent memory. Only θ_{Mem} is optimised on the public training split with teacher-forced answer loss; the encoder and decoder remain frozen throughout. No method may use an external summarizer, a different retriever, or a different pretrained model.

Implementation details. The frozen backbone is Flan-T5-base (248M parameters) in bfloat16. All memory adapters are trained with AdamW (learning rate 10^{-4}, weight decay 10^{-2}, linear warmup of 200 steps, gradient norm clipped at 1.0) for 10 epochs with batch size 4 and gradient accumulation 4 (effective batch 16). Write-rule updates are detached from the computation graph; truncated backpropagation through time uses a window of k=8 turns. Shared memory hyper-parameters are: bank size n_P=64, write decay γ=0.95. M.4 uses associative dimension d_h=256; M.6 uses S=64 slots with top-k=8 writes per turn. All experiments use a single seed (42) and a single NVIDIA GPU.

The primary benchmark is **LoCoMo** [5], a long-term conversational-memory dataset with explicit QA supervision and annotated evidence turns. Those evidence annotations are the key ingredient for the rebuilt evaluation: they let us place every question at a precise lag in the past. We do *not* use MSC [4] as a primary score in this paper, because MSC lacks per-answer evidence locations and therefore cannot support a clean absolute forgetting curve.

6.3 Forgetting-Curve Evaluation

We evaluate one quantity only: the *absolute forgetting curve*. For a LoCoMo QA pair q with gold answer y_q and annotated evidence turns E_q, let

$$\ell_q = T - \min E_q \tag{35}$$

denote the lag in turns from the oldest required fact to the end of the conversation.

For each method m, we compute two answers to the same question:

$$\hat{y}_q^{\text{mem}}(m) = \text{answer from method } m \text{ after writing the conversation into } P, \tag{36}$$

$$\hat{y}_q^{0}(m) = \text{answer from the same trained method with its persistent state forced to zero.} \tag{37}$$

The absolute retained-memory score of question q is then

$$R_q(m) = \max\Big(0,\ F1\big(\hat{y}_q^{\text{mem}}(m), y_q\big) - F1\big(\hat{y}_q^{0}(m), y_q\big)\Big). \tag{38}$$

This is *not* baseline-relative. It is an absolute, method-internal quantity: how much answer quality disappears when that method's own memory is ablated. Importantly, $R_q(m)$ is a *differential F1 gain*, not a percentage of facts recalled. A score of, say, 3.83 % means that ablating memory reduces the token-level F1 by 3.83 percentage points; it does not mean the model remembers only 3.83 % of the stored information. The stateless baseline has no persistent state at all, so

$$R_q(\text{M.0}) = 0 \qquad \text{for all questions } q. \tag{39}$$

We bucket lags into five logarithmic ranges, $[0, 32)$, $[32, 64)$, $[64, 128)$, $[128, 256)$, and $[256, \infty)$ turns, average $R_q(m)$ inside each bucket, and then fit a weighted non-increasing isotonic curve across buckets. The isotonic fit suppresses finite-sample noise while preserving the forgetting prior that retained information should not improve as lag increases. A stronger method therefore has a higher intercept at short lag and a slower downward decay.

6.4 Results

Figure 9 is the primary empirical result. The short-lag height measures write effectiveness: how much conversation-specific information is available immediately after storage. The slope measures resistance to overwrite and interference: how slowly that information decays as the relevant evidence recedes into the past.

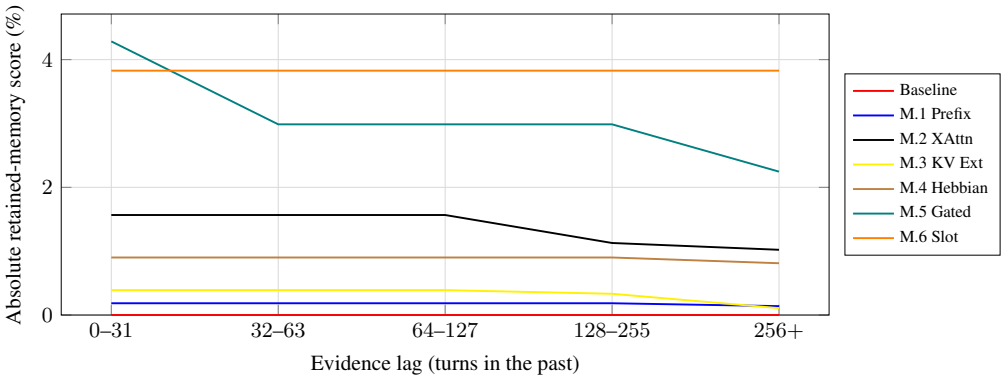

Figure 9: Absolute forgetting curves on LoCoMo. Each point is the retained-memory score from Eq. (38), averaged within a lag bucket and then smoothed by a weighted non-increasing isotonic fit. The baseline is identically zero because it has no persistent state. Higher and flatter curves indicate stronger memory.

Interpretation. The rebuilt evaluation isolates persistent memory itself rather than generic question-answering ability. A method receives credit only for answer quality that vanishes when its own persistent state is ablated. The baseline is therefore exactly flat at zero, while trained memory methods show positive short-lag retention followed by gradual decay. Because the score is a *differential* F1 gain (not a recall rate), even single-digit values represent meaningful memory contribution: the model's overall F1 may be much higher, but only the portion attributable to persistent memory is counted.

The figure separates two properties that were conflated by the previous multi-metric protocol. Short-lag height measures whether a method writes useful content into memory at all. Long-lag decay measures whether that content survives overwriting and interference. A stronger architecture is one whose curve stays higher for longer.

On the plotted smoothed curves, the methods separate cleanly into tiers. M.6 Slot is the best overall architecture: it retains 3.83% in every lag bucket, so it has both the highest mean forgetting-curve score and the strongest long-lag memory. M.5 Gated is second overall and shows the strongest immediate write signal, reaching 4.29% in the 0–31 bucket, but it decays to 2.25% by the 256+ bucket, indicating stronger overwrite sensitivity than M.6. M.2 XAttn forms a clear middle tier (1.57% at short lag, 1.02% at long lag), while M.4 Hebbian is weaker in absolute magnitude but relatively stable (0.90% to 0.81%). M.3 KV Ext and M.1 Prefix remain close to the zero line throughout; among trained methods, M.1 Prefix is the weakest. The overall ordering on the plotted forgetting-curve score is therefore M.6 Slot > M.5 Gated > M.2 XAttn > M.4 Hebbian > M.3 KV Ext > M.1 Prefix > Baseline.

The fact that four of six trained adapters produce non-trivial retained-memory curves answers the secondary question posed in the introduction: the frozen decoder's cross-attention does possess sufficient representational slack to attend usefully to memory entries projected by a trained adapter—the bottleneck is not the decoder itself but the quality of the write and read pathway.

7 Discussion

7.1 Why training is necessary

If persistent memory is built entirely from frozen encoder outputs and exposed to the decoder without learned projections, the decoder receives states that were never optimised for selective long-range retrieval. The failure is structural: as more history is concatenated or cached, useful entries compete with irrelevant ones inside the same softmax, attention mass disperses, and the contribution of any single remembered fact shrinks. The frozen encoder manifold $\mathcal{M}_E$ matches the pre-trained cross-attention interface, but it provides no mechanism for deciding what to retain, compress, or foreground across long lags.

The forgetting-curve results confirm this analysis. Methods that learn both a write rule and a read path (M.5 Gated, M.6 Slot) achieve the highest retained-memory scores, whereas methods whose projections are closer to the raw encoder output (M.1 Prefix, M.3 KV Ext) barely separate from the zero baseline. Training is therefore necessary: the adapter must learn to map persistent state back into a representation the frozen decoder can use.

7.2 Limitations

All six methods are instantiated on a single frozen encoder–decoder backbone (Flan-T5-base, 248 M parameters). Whether the same architectural principles transfer to decoder-only, encoder-only, or larger-scale models must be validated separately. Performance will depend on memory budget, the amount of supervision available for training θ_{Mem}, and the temporal structure of the evaluation benchmark. The evaluation centres on LoCoMo because MSC [4] lacks explicit evidence annotations; extending the absolute forgetting-curve protocol to open-domain multi-session chat will require additional supervision or a different probing design. The present study also isolates six trained adapters under a single frozen backbone; it does not yet test richer structured latent-memory systems or partially unfrozen backbones under the same protocol.

7.3 Broader implications

End-to-end training and conversational learning. Here only the memory adapter is trained. The full potential of persistent latent-space memory would be realised when the *entire* LLM is trained end-to-end with its memory bank, learning simultaneously what to store and how to use it. Such a system would process each turn with a short, fixed-length prompt supplemented by persistent memory, rather than re-ingesting an ever-growing context window—sidestepping the quadratic attention cost that dominates current long-context models. More broadly, persistent memory opens the door to

conversational learning: every interaction updates the bank, and the model becomes more informed with each turn, driven by ordinary dialogue rather than curated datasets or reward signals.

Scalability. A latent memory bank is a compact numerical array whose capacity can grow without increasing the per-turn inference cost of the backbone. Text-level memory systems, by contrast, must retrieve, serialise, and re-tokenise growing text stores, so their cost scales with memory size. Persistent latent memory is therefore inherently cheaper and more scalable.

Latent-space memory as a cognitive substrate. Biological brains do not retain verbatim transcripts; they maintain distributed, continuously updated representations that support recognition, abstraction, and generalisation. Persistent latent-space memory mirrors this organisation more closely than text-level retrieval, and connects naturally to neuroscience-inspired architectures such as complementary learning systems [10] and attention-coupled lateralised memory [6]. Because LLMs already represent knowledge as continuous activations, a persistent memory that operates in the same latent space is a more natural substrate for core cognitive operations—reading, updating, generalisation, and compositional extension—than a symbolic or textual buffer.

8 Conclusion

Recent work on attention-coupled latent memory has shown that persistent banks can support richer structured phenomena such as inhibitory lateralization and brain-inspired gating [6, 7]. Here we isolate a simpler but practically important setting: persistent memory that lives entirely in the *latent space* of a frozen encoder–decoder LLM, with only a small trainable memory adapter.

We evaluated six such adapters, spanning three injection points (before encoder, between encoder and decoder, inside decoder) and four write mechanisms (attention-coupled, Hebbian, gated cross-attention, sparse slot addressing). Under the absolute forgetting-curve evaluation on LoCoMo, the baseline is exactly zero, all trained methods retain some conversation-specific information, M.6 Slot is best overall, M.5 Gated has the strongest short-lag write signal but decays more strongly, and M.1 Prefix is the weakest trained adapter. The comparison therefore identifies slot-based persistent memory as the strongest of the six minimal frozen-backbone adapters tested here.

The broader implication is that latent-space persistent memory in frozen LLMs is not one mechanism but a design space. Because the memory bank holds continuous representations rather than text, every design choice (write rule, read path, capacity) has measurable consequences on the forgetting curve—consequences that would be invisible in a text-level memory system. Some adapters mainly improve immediate write effectiveness, while others improve long-lag resistance to overwrite and interference. The forgetting-curve view makes those tradeoffs explicit and provides a direct criterion for future work on richer latent-space memory architectures.

Acknowledgements

The author thanks Inha University in Tashkent for research support. This work reflects the author's ongoing inquiry into nature and human cognition.

References

[1] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 21(140):1–67, 2020.

[2] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2024.

[3] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: enhancing large language models with long-term memory. In *AAAI*, 2024.

[4] Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. In *ACL*, 2022.

[5] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the ACL*, 2024.

[6] Hong Jeong. Inhibitory cross-talk enables functional lateralization in attention-coupled latent memory. *arXiv:2603.03355 [q-bio.NC]*, 2026. `https://arxiv.org/abs/2603.03355`.

[7] Hong Jeong. A miniature brain transformer: Thalamic gating, hippocampal lateralization, amygdaloid salience, and prefrontal working memory in attention-coupled latent memory. *arXiv:2603.07217 [q-bio.NC]*, 2026. `https://arxiv.org/abs/2603.07217`.

[8] Endel Tulving. Episodic and semantic memory. In E. Tulving and W. Donaldson, editors, *Organization of Memory*, pages 381–403. Academic Press, 1972.

[9] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

[10] Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? Complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.

[11] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *ACL–IJCNLP*, 2021.

[12] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, Roman Ring, Eliza Rutherford, Serkan Cabi, Tengda Han, Zhitao Gong, Sina Samangooei, Marianne Monteiro, Jacob Menick, Sebastian Borgeaud, Andrew Brock, Aida Nematzadeh, Sahand Sharifzadeh, Mikolaj Binkowski, Ricardo Barreira, Oriol Vinyals, Andrew Zisserman, and Karen Simonyan. Flamingo: a visual language model for few-shot learning. In *NeurIPS*, 2022.

[13] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *ICLR*, 2022.

[14] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *ICML*, 2021.

[15] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing machines. *arXiv preprint arXiv:1410.5401*, 2014.